

Geospatial with Parquet & DuckDB





Who am I?



Florent Fougères

Geographer and geomatics engineer, specialized in GIS development

French GIS Developer at [Oslandia](#)



@florentfougeres



@florentfougeres



@florentfgrs@mastodon.social



01



Parquet

Parquet format ?

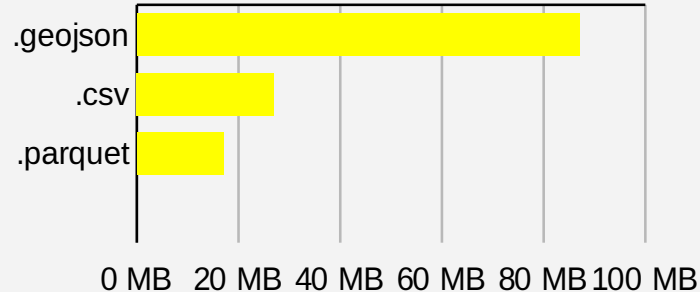
- Open Source file format
- Driven by Apache Foundation
- Store & Access large amounts of data
- Column-oriented architecture
- Efficient compression
- + Geoparquet for geometries
- Folder of Parquet files => a single dataset
 - (Files processed in chunks for parallelism and efficient handling of large data)



Parquet Benchmark



Data sets for all localities worldwide with more than 1,000 inhabitant available [here](#)



Parquet to DuckDB

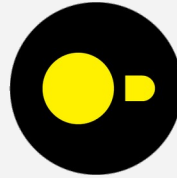
- Parquet format = exchange format.
- Not a working / processing format.
- Optimized database system required
- Open data format (Parquet)
+ in-process database (DuckDB)
= maximum efficiency



DuckDB



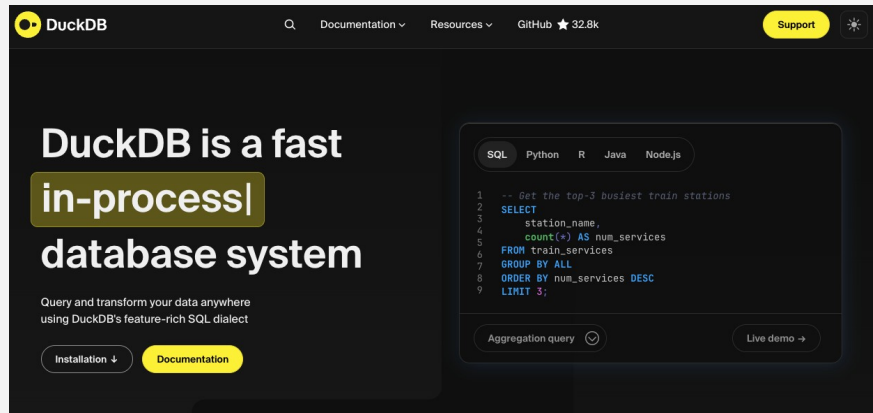
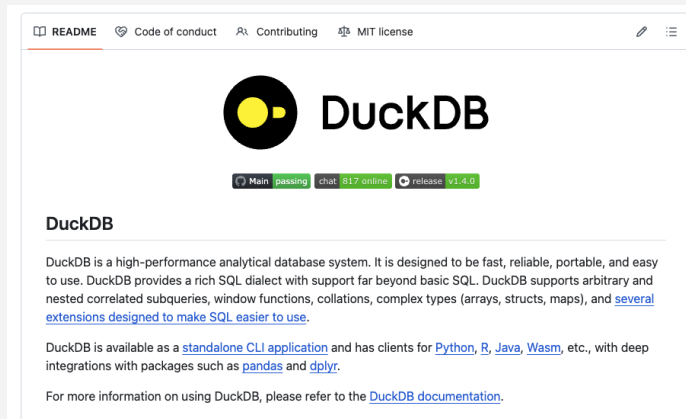
02

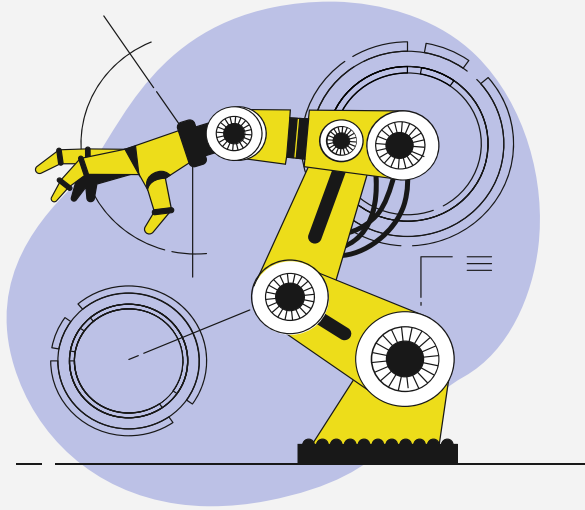


DuckDB

DuckDB ?

- Relational database management system
- Written in C++
- MIT License
- Database in portable file format
- Open source, of course !
- Currently stable in version 1.4





Why ?

- Serverless
- Quick set up
- Database in portable file format
- SQL language
- Reading files directly in from clause
- Spatial extension with GIS functions



Spatial extension

- Lot of spatial functions based on GEOS.
- Same names as PostGIS
- Spatial indexing
- Reads and writes lot of spatial files
- GDAL Drivers

```
INSTALL spatial ;  
LOAD spatial ;
```



The screenshot shows the DuckDB Docs website with a dark theme. The left sidebar contains a navigation menu with categories like Installation, Documentation, and Core Extensions. The main content area is titled 'DOCUMENTATION / CORE EXTENSIONS / SPATIAL' and 'Spatial Functions'. It includes a 'Function Index' table with columns for 'Function' and 'Summary'. The table lists various spatial functions such as `DuckDB_Proj_Version`, `ST_Affine`, `ST_Area`, `ST_Area_Spheroid`, `ST_AsGeoJSON`, `ST_AsHEXKB`, `ST_AsSVG`, `ST_AsText`, `ST_AsWKB`, `ST_Azimuth`, `ST_Boundary`, `ST_Buffer`, `ST_BuildArea`, `ST_Centroid`, `ST_Collect`, `ST_CollectionExtract`, and `ST_ConcaveHull`. A right sidebar titled 'IN THIS ARTICLE' lists the functions covered in the article.

Function	Summary
<code>DuckDB_Proj_Version</code>	Returns a text description of the PROJ library version that this instance of DuckDB was compiled against.
<code>DuckDB_Proj_Version</code>	Returns a text description of the PROJ library version that is being used by this instance of DuckDB.
<code>ST_Affine</code>	Applies an affine transformation to a geometry.
<code>ST_Area</code>	Compute the area of a geometry.
<code>ST_Area_Spheroid</code>	Returns the area of a geometry in meters, using an ellipsoidal model of the earth.
<code>ST_AsGeoJSON</code>	Returns the geometry as a GeoJSON fragment.
<code>ST_AsHEXKB</code>	Returns the geometry as a HEXWKB string.
<code>ST_AsSVG</code>	Convert the geometry into a SVG fragment or path.
<code>ST_AsText</code>	Returns the geometry as a WKT string.
<code>ST_AsWKB</code>	Returns the geometry as a WKB (Well-Known-Binary) blob.
<code>ST_Azimuth</code>	Returns the azimuth (a clockwise angle measured from north) of two points in radians.
<code>ST_Boundary</code>	Returns the "boundary" of a geometry.
<code>ST_Buffer</code>	Returns a buffer around the input geometry at the target distance.
<code>ST_BuildArea</code>	Creates a polygonal geometry by attempting to "fill in" the input geometry.
<code>ST_Centroid</code>	Returns the centroid of a geometry.
<code>ST_Collect</code>	Collects a list of geometries into a collection geometry.
<code>ST_CollectionExtract</code>	Extracts geometries from a GeometryCollection into a typed multi geometry.
<code>ST_ConcaveHull</code>	Returns the 'concave' hull of the input geometry, containing all of the source inputs points, and which can be used to create polygons from points. The ratio parameter dictates the level of concavity; 1.0 returns the convex hull; and 0 indicates to return the most concave hull possible. Set allowHoles to a non-zero value to allow holes in the input geometry.

Performance



- OLAP (Online Analytical Processing) versus OLTP
- Column-oriented architecture
- Processing large volumes of data
 - multiple gigabytes, multiple million lines.



Performance



- Overtures Maps Geojson data covering 13.1 million buildings
- Geojson file : 6.3 GB.
- (remember : OLAP only, no OLTP support)

	Integration	Buffer calculation	Adding a column and calculating the area
DuckDB	4'	0'38"	0'2"
PostGIS	5'30	1'54"	2'58"



Limitations



- No projection support
- Single-user, local
- Read mode => database locked
- No rights and user management
- Young project, evolving fast
- OLAP, not OLTP
 - frequent writes, concurrent transactions, durability, HA...



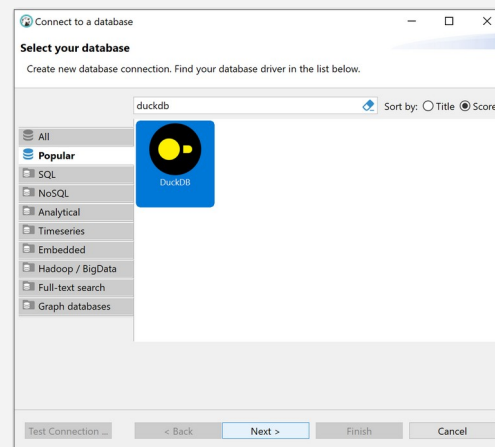
Using DuckDB



- CLI for all OS : Linux, Mac OS, Windows
- Package available in many languages.



- In open source database IDE: Dbeaver
- DuckDB Wasm → Web assembly version
- And in many tools, libraries, and resources thanks to a large number of contributors on this Github repository



03



GIS use

DuckDB with QGIS : QduckDB Plugin



- Initial funded by IFREMER
- Only read mode for now.
- Plugin
- Code repository

The screenshot shows the QGIS Plugin Repository interface. At the top, there's a navigation bar with links for About, Resources, Community, Download, Donate, and a user icon. Below this is a dark blue header with Home, All Plugins, Categories, and Metrics. A search bar and a Login button are on the right. The main content area features a sidebar on the left with links to Home, All Plugins, Categories, Metrics, Documentation, and Plugin Tags. The central part displays the QDuckDB plugin details, including its icon (a black circle with a yellow duck head), Plugin ID (3192), a description stating it adds a new data provider for DuckDB, a 5-star rating from 31 votes, and a 'Download latest' button. Below this are tabs for About, Details, and Versions. The 'About' tab is active, showing text about the plugin's purpose and installation instructions for Windows users, including a link to more information in the documentation.



<https://video.osgeo.org/w/1YGiDEggK29RAo9TmtCjc1>

Spatial processing, format conversions



Spatial layer from CSV file with x and y columns



```
CREATE TABLE point_layer AS  
SELECT *, ST_Point(x, y) as geom  
FROM read_csv("/path/to/data.csv")
```



Spatial processing, format conversions



Convert Shapefile to GPKG

```
COPY (  
    SELECT *  
    FROM ST_Read('path/to/input.shp')  
) TO 'path/to/output.gpkg' WITH (FORMAT GDAL, DRIVER 'GPKG');
```



Spatial processing, format conversions



Shapefile from parquet file with BBOX filter

```
COPY (  
  SELECT *  
  FROM read_parquet('path/to/buildings.parquet')  
  WHERE ST_Within(  
    geom,  
    ST_MakeEnvelope(4.2, 50.75, 4.6, 50.95, 4326)  
  )  
) TO 'path/to/brussel_buidings.shp' WITH (FORMAT GDAL, DRIVER 'ESRI Shapefile');
```



Vector tile server



- Vector tiles generation with DuckDB 1.4+
- Example by Max Gabriellsson :

<https://gist.github.com/Maxxen/37e4a9f8595ea5e6a20c0c8fbbefe955>





Thank you for your attention!

